PROJECTO 1
EFEITOS ESPECIAIS (ßeta)

INTRODUÇÃO E OBJECTIVOS

A finalidade deste projecto passa por consolidar os conhecimentos de programação em Python adquiridos até agora, através da implementação de dois programas de natureza diferente. Um objectivo secundário deste projecto consiste em introduzir a programação orientada para sistemas recorrendo, para tal, a bibliotecas desenvolvidas por terceiros.

Na primeira parte do projecto, deve implementar um conjunto de algoritmos para produzir "efeitos especiais" em texto. Na segunda parte, pretende-se que implemente um dos utilitários seguintes: programa de encriptação; listagem de conteúdo de directoria em árvore; obtenção de endereços de rede da camada MAC e programa que detecta semelhanças entre ficheiros.

Alguns elementos do projecto são considerados "extra", sendo resumidos na secção **AVALIAÇÃO**. Nesta secção encontra também uma explicação sobre o que se pretende dizer com "extra" no contexto deste projecto.

PARTE I - EFEITOS ESPECIAIS

Nesta parte deve desenvolver um programa em Python para exibir o texto introduzido na linha de comandos de acordo com determinados "efeitos especiais" (ver à frente). O seu programa deve ser invocado da seguinte forma (propositadamente, neste e noutros programas deste enunciado omitimos o interpretador; ou seja, consonte a sua plataforma poderá ter que prefixar o script com `python` ou `python3`):

```
$ efeitos.py [-i INTERVALO] [-d DIM] palavra1 [palavra2] ... [palavraN]
```

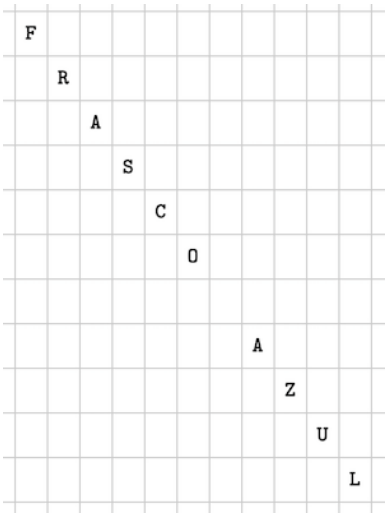
Depois aplica cada um dos efeitos em baixo descritos ao texto que resulta da concatenação de `palavra1`, `palavra2`, etc. Mais à frente no enunciado encontra o significado das opções `-i` e `-l`.

Os caracteres do texto (que resultam da concatenação de todas as palavras) devem ser exibidos individualmente.

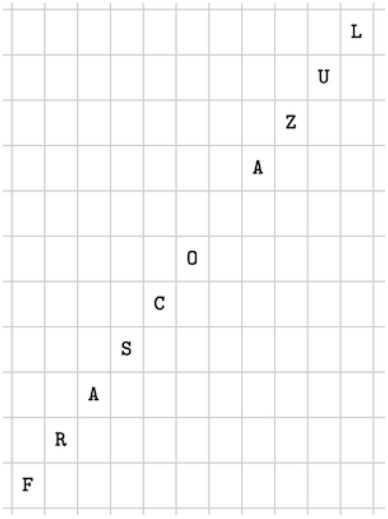
Segue-se uma demonstração dos efeitos pretendidos. A não ser que exista alguma indicação noutro sentido, assuma que o programa foi invocado da seguinte forma:

```
$ efeitos.py FRASCO AZUL
```

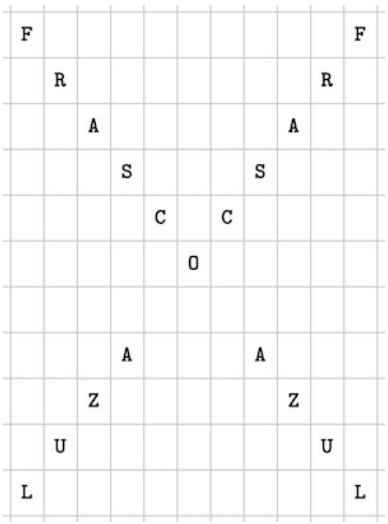
Efeito 1- Diagonal Esquerda



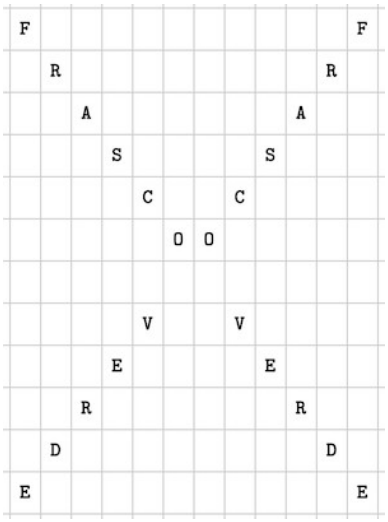
Efeito 2 - Diagonal Direita, Texto Invertido



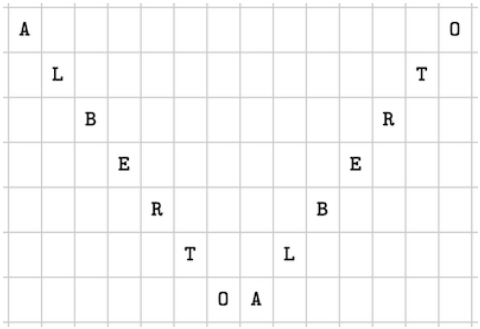
Efeito 3 - Diagonais Cruzadas (num. cars. ímpar)



Efeito 3 - Texto: "FRASCO VERDE" (num. cars. par)



Efeito 4 - Em V, Texto: "ALBERTO



Efeito 5 - Em Escada, Palavras por Ordem Inversa. Texto: `"FRASCO AZUL ABERTO"`

A	B	E	R	T	O		
	A	Z	U	L			
		F	R	A	S	C	O

Efeito 6 - Texto Deslizante em Ciclo ao Longo da linha. Texto: `"ALBERTO"`

t	*Linha de texto exibida*												
0.0	A	L	B	E	R	T	O						
0.5		A	L	B	E	R	T	O					
1.0			A	L	B	E	R	T	O				
	. . .												
3.5	O							A	L	B	E	R	T
4.0	T	O							A	L	B	E	R

A coluna da esquerda representa o tempo decorrido desde o início do efeito. Estamos a assumir um intervalo de tempo entre exibição de linhas de 0.5s. Todas as quadriculas livres são preenchidas com espaços (caractere ' ') Note que o texto NÃO deve ser exibido em escada como dá a entender esta imagem estática. Cada uma das linhas fica "por cima" da anterior, dando a ilusão que o texto se está a mover para a direita.Veja também a demonstração que acompanha este enunciado.

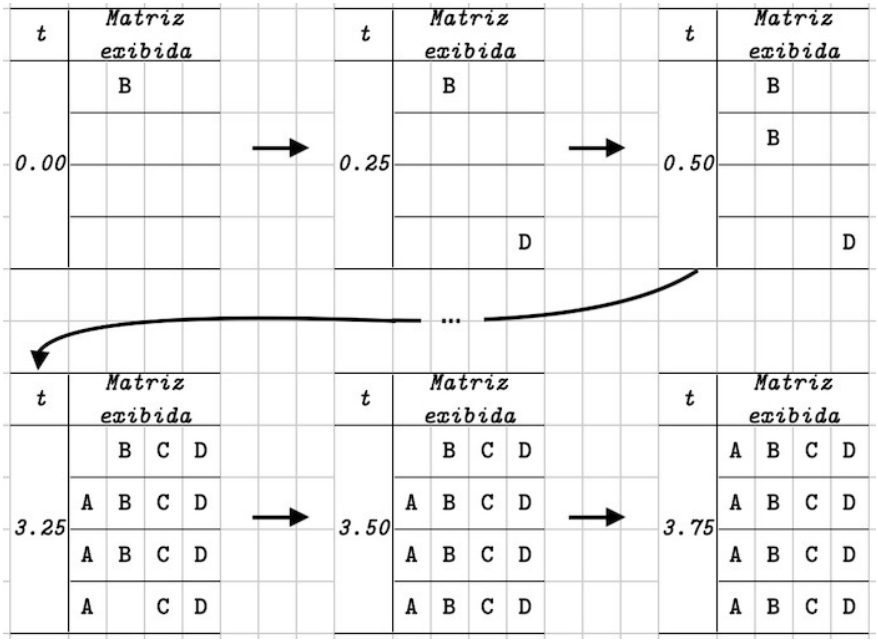
Efeito 7 - Destapa Caracteres em Posições Aleatórias. Texto: `"ALBERTINA"`

t	*Linha de texto exibida*								
0.00							I		
0.25			B				I		
0.50			B		R		I		
...	. . .								
1.50		L	B	E	R		I	N	A
1.75	A	L	B	E	R		I	N	A
2.00	A	L	B	E	R	T	I	N	A

t	*Linha de texto exibida*								
0.00		L							
0.25		L				T			
0.50		L			R	T			
...	. . .								
1.50	A	L		E	R	T		N	A
1.75	A	L		E	R	T	I	N	A
2.00	A	L	B	E	R	T	I	N	A

Em cima vemos duas possibilidades para o texto indicado. Estamos a assumir um intervalo de tempo entre exibição de linhas de 0.25s. Quadriculas livres preenchidas com espaços. Cada uma das linhas fica "por cima" da anterior, ou seja, em cada instante só vemos uma linha. No instante final (2.00, neste exemplo) vemos uma linha com o texto completo. Veja também a demonstração que acompanha este enunciado.

Efeito 8 - Destapa Matriz em Posições Aleatórias. Texto: `"ABCD"`



Em cima, de forma resumida, tentamos ilustrar a exibição do texto. Quadriculas livres são preenchidas com espaços. Assume-se um intervalo de tempo entre exibição de matrizes de 0.25s. Veja também a demonstração que acompanha este enunciado.

CONSIDERAÇÕES E RECOMENDAÇÕES

Para o efeito da linha deslizante, assuma que cada linha tem 40 caracteres de largura e que o intervalo entre exibição de linhas (para que seja visível o efeito deslizante) é 0,1 segundos. Quando "está sob este este efeito", o programa termina apenas quando o utilizador o interrompe com `CTRL+C` (pergunte ao formador como detectar a inserção do `CTRL+C`). Como vai ter que inserir o texto na linha de 40 caracteres, sugere-se que crie uma lista com 40 caracteres *(qual o caractere com que deve inicializar a lista?)* e insira o texto nesta lista nas posições apropriadas. Depois, antes de exibir esta lista, deve transformá-la numa string com o método **`str.join`**. Veja o seguinte exemplo:

```
txt = 'FRASCO AZUL'
i = 5
line = ['.'] * 40
line[i:i + len(txt)] = txt
print(''.join(line))

i += 1
line = ['.'] * 40
line[i:i + len(txt)] = txt
print(''.join(line))
```

Para os restantes efeitos que dependem de um temporizador para exibir o texto, assuma também que intervalo de tempo é 0,1 segundos. Todavia, este intervalo pode ser parametrizado como indicado à frente.

Para "temporizar" a execução do código em Python pode utilizar a função `sleep` definida no módulo time. Esta função recebe um ***float*** com indicação do intervalo de tempo em segundos. Por exemplo, `sleep(1)` ou `sleep(1.0)` suspendem o processo durante 1s, ao passo que `sleep(1.5)` suspende durante um segundo e meio, e `sleep(0.001)` suspende durante um milisegundo. Para apagar o ecrã, sugere-se a invocação do comando `clear` em sistemas do tipo Unix (macOS, Linux, FreeBSD, etc.) ou o comando `cls` em sistemas Windows. Eis um exemplo que utiliza estes mecanismos:

```
import time
import subprocess

subprocess.run(['clear'])        # substituir subprocess.run(['clear'])
print("ALBERTO...")              # por subprocess.run(['cls'], shell=True) em
time.sleep(0.5)                  # Windows
subprocess.run(['clear'])
print(".ALBERTO..")
time.sleep(0.5)
subprocess.run(['clear'])
print("..ALBERTO.")
time.sleep(0.5)
subprocess.run(['clear'])
```

Como alternativa a apagar o ecrã, pode simular este efeito escrevendo sempre na linha inicial. Isto é possível se chamar a função print com `end = '\r'` (em vez, de utilizar o valor por omissão `'\n'`). Exprimente:

```
import time
print("ALBERTO...", end='\r')
time.sleep(0.5)
print(".ALBERTO..", end='\r')
time.sleep(0.5)
print("..ALBERTO.", end='\r')
time.sleep(0.5)
print()
```

Utilize o módulo `random` para os efeitos que "destapam" posições do texto de forma aleatória. Uma função que pode ser particularmente útil é a função `random.shuffle`. Esta função "baralha" uma lista de objectos. Por exemplo, dada a string `txt`, eis como gerar uma lista com posições aleatórias que depois pode utilizar para aceder a `txt` de forma aleatória:

```
import random
txt = 'ALBERTO'
random_positions = list(range(len(txt)))
random.shuffle(random_positions)
# agora podemos aceder a txt utilizando as posições na lista random_positions
```

De um modo geral, exiba um separador apropriado entre todos os efeitos: faça uma pausa entre efeitos, seguida de um "apagar" de ecrã, para que seja visível o efeito em questão.

INTERFACE COM O UTILIZADOR

O programa deve aceitar o parâmetro `-i` com o valor do intervalo de actualização em segundos. O valor por omissão deste parâmetro deve ser 0.1s. Aceita também a opção `-d` que permite especificar a dimensão da linha para o efeito deslizante. 40 é o valor por omissão deste parâmetro. Eis como pode exibir `ALBERTO` a deslizar para a direita com um intervalo de actualização de 0.25s, numa linha com 37 espaços:

```
$ efeitos.py -i 0.25 -d 37 ALBERTO
```

O valor do parâmetro `-i` é utilizado em todos os efeitos relevantes e não apenas no efeito deslizante.

O programa deve começar por exibir o menu indicado à frente exactamente como demonstrado à frente. As opções numeradas exibem o efeito correspondente em ecrã à parte, isto é, o programa deve apagar o ecrã, exibir o efeito pretendido e depois "fazer uma pausa". Após a pausa, o menu é exibido novamente.

A opção T leva a que todos os efeitos sejam exibidos. Deve colocar uma pausa seguida de um apagar de ecrã entre efeitos.

A introdução de uma opção não contemplada no menu deve ser assinalada ao utilizador como sendo uma opção inválida.

```
**************************************************
*                                                *
*  EFEITO                                        *
*                                                *
*    1 - Diagonal Esquerda                       *
*    2 - Diagonal Direita, Texto Invertido       *
*    3 - Diagonais Cruzadas                      *
*    4 - Em V                                    *
*    5 - Escada, Palavras Ordem Inversa          *
*    6 - Deslizante                              *
*    7 - Destapa Posições Aleatórias             *
*    8 - Destapa Matriz                          *
*    T - Todos                                   *
*    E - Encerrar                                *
*                                                *
**************************************************

  OPÇÃO>
```

PARTE II - PROGRAMA À ESCOLHA

Implemente um ou mais dos dos utilitários em baixo descritos, de acordo com as instruções do formador.

CIFRA DE VIGENÈRE

A cifra de Vigenère é um método de encriptação equivalente a aplicar uma série de cifras de César em sequência, cada cifra com diferentes valores de deslocamento. Estes deslocamentos são estabelecidos por uma palavra-passe. O primeiro byte da palavra-passe indica o deslocamento a aplicar ao primeiro byte da sequência, o segundo byte da palavra-passe indica o deslocamento a aplicar ao segundo byte da sequência, e assim sucessivamente.

Esta encriptação poder ser feita por meio de uma tabela designada de Tábua de Vigenère (Vigenère Square) ou por Tabula Recta. O algoritmo e a utilização desta estrutura de dados são descritos em pormenor em https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher#Description .

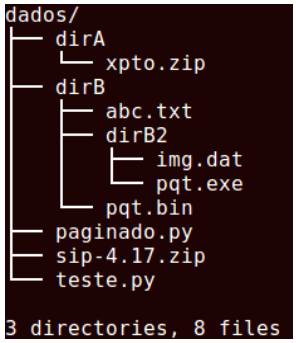
O comando **`vcypher`** permite encriptar e desencriptar um ficheiro com a cifra de Vigenère. A sintaxe do comando deve ser a seguinte:

```
$ vcypher.py -e [-p PASSWORD] [-o OUT_FILE] FILE
$ vcypher.py -d -p PASSWORD [-o OUT_FILE] FILE
```

Com a opção `-e` o **`vcypher`** encripta o ficheiro dado por `FILE` utilizando para tal a password passada para a opção `-p`. No caso de não ser indicado nenhuma com a opção `-p`, o programa gera uma password aleatória com 6 caracteres legíveis da gama ASCII, devendo no final exibir na saída padrão qual o valor dessa palavra-passe. Com a opção `-d`, o vcypher desencripta o ficheiro `FILE`. Neste caso, a palavra-passe é obrigatória. Este comando deve suportar as opções longas `--encrpyt`, `--decrypt`, `--output-file` e `--password` como alternativa às opções curtas `-e`, `-d`, `-o` e `-p`. O programa deve confirmar junto do utilizador se o ficheiro a encriptar/desencriptar já existe antes de gravar por cima.

Para iludir possíveis intrusos, a extensão do ficheiro codificado deve ser `.pdf`. Utilize a biblioteca **docopt** ou **argparse** para ler os argumentos da linha de comandos.

TREEP



Este comando, um clone simplificado do comando `tree` utilizado em Linux, lista o conteúdo de uma directoria num formato semelhante ao indicado na figura seguinte. A sintaxe de invocação é a seguinte:

```
$ treep.py [-df] [-L <level>] [-H] <caminho_dir>
```

Consulte a ajuda do comando `tree` (`$ man tree` em Linux, também disponível na Web) para obter o significado das opções `-d`, `-f` e `-L`. Deve utilizar a biblioteca **argparse** para análise dos argumentos da linha de comandos. Como elemento extra, pode

desenvolver a opção `-H` para exibir a listagem em HTML/CSS. Também aqui a saída deve similar à do `tree`.

SYMS

Vamos agora fazer um programa para detectar ficheiros com "semelhanças". Através da linha de comandos, o seu programa recebe um caminho para uma directoria e "desce" essa directoria procurando por ficheiros que apresentem uma determinada semelhança. No final exibe uma listagem com os grupos de ficheiros semelhantes.

De seguida, precedidas da respectiva opção da linha de comandos, indicamos as semelhanças a detectar:

- `-c, --contents`: detecta ficheiros com conteúdo binário igual
- `-n, --name`: detecta ficheiros com o mesmo nome (incluindo a extensão)
- `-e, --extension`: detecta ficheiros com a mesma extensão
- `-r PATTERN, --regex==PATTERN`: detecta ficheiros cujo o nome verifica a expressão regular dada por `PATTERN`; os ficheiros são depois agrupados pela ocorrência concreta do padrão

A sintaxe do comando deve ser a seguinte:

```
$ syms.py [-c] [-n] [-e] [-r PATTERN] [caminho_dir]
```

Utilize a biblioteca **docopt** para leitura das opções da linha de comandos. Se mais do que uma opção for especificada, então o seu programa deve exibir uma listagem por opção. Se nenhuma opção for especificada, o programa assume a opção `-n`. O valor por omissão para `caminho_dir` é a directoria corrente.

Sugestões:

1. Utilize `os.walk` para percorrer árvore de directorias dentro do caminho fornecido

2. No caso da detecção de ficheiros com o mesmo conteúdo, para evitar ter que comparar ficheiros, processo dispendioso em termos de CPU e de utilização de memória, utilize `hashlib.md5` para obter de forma eficiente um resumo de cada ficheiro. É praticamente impossível encontrar dois ficheiros para os quais se obtenham resumos idênticos (ou seja, nestas circunstâncias, e para efeitos práticos, um *hash* é um resumo unívoco).

Consultar: https://docs.python.org/3/library/os.html?#os.walk
https://docs.python.org/3/library/hashlib.html#module-hashlib
https://www.tutorialspoint.com/python/python_reg_expressions.htm
https://docs.python.org/3/howto/regex.html
https://towardsdatascience.com/learn-enough-python-to-be-useful-argparse-e482e1764e05
https://docs.python.org/3/library/argparse.html

AVALIAÇÃO

No contexto deste projecto, um elemento "**extra**" é um componente ou funcionalidade cuja cotação (ver tabela em baixo) é substancialmente inferior à de outros componentes ou funcionalidades de dificuldade semelhante. É possível ter uma boa classificação neste projecto não realizando nenhum dos elementos extras.

Elemento	Cotação Máxima (0..20)
Efeitos	16
Programa Parte II	3
Realização de outro programa Parte II	1

Como incentivo, a classificação das funcionalidades extra pode contribuir até 2 valor(es) para a nota do teste escrito e até 1 valor(es) para a nota do próximo projecto. O projecto deve ser resolvido em grupos de dois formandos. Excepcionalmente, poderá ser realizado por grupos com outras dimensões.

Deve também elaborar um relatório (ver secção **Prazo e Entregas**) que valerá no máximo 20% da cotação do projecto.

PRAZOS E ENTREGAS

O trabalho final deve ser entregue até às 23h59m do dia 07/05/2026. Um atraso de N dias na entrega levará a uma penalização dada pela fórmula $0.5 \times 2^{(N-1)}$ ($N > 0$). Poderão ser marcadas sessões do acompanhamento do trabalho. A classificação final poderá ser influenciada pelo progresso demonstrado nessas sessões de acompanhamento.

Caso o formador não indique outro procedimento, na data final deverá entregar o seguinte:

1. Um **ZIP** contendo o projecto do VS Code. O ZIP deve possuir os nomes dos membros do grupo sem acentos e separados por _ (underscore). Supondo que o projecto foi realizado pela Ana Alves e pelo Jose Sá, o ZIP deve ser designado de `AnaAlves_JoseSa.zip`.

Este ZIP deve conter uma única directoria:

`Projecto1` : directoria com o Workspace do VS Code.

1.1 Por seu turno a directoria `Projecto1` deve conter as seguintes sub-directorias:

`.vscode`: directoria com as definições do VS Code

`src`: directoria com o código fonte

`docs`: documentação do projecto mais o relatório (ver em baixo)

1.2 O conteúdo da directoria `/Projecto1/src` deve ser o seguinte:

Efeitos especiais: `efeitos.py`

Cifra de Vigenère: `vcypher.py`

Treep: `treep.py`

Syms: `syms.py`

NOTA: *Não colocar no ZIP o ambiente virtual (directoria `.venv`) nem a directoria do Git (directoria `.git`)*

2. Em alternativa ao ZIP poderá enviar o ficheiro `AnaAlves_JoseSa.txt` (ver ponto anterior) contendo uma ligação para o repositório privado no **GitHub**. Lembre-se que deverá convidar o formador a participar no repositório antes da data da entrega. Além de que não poderá introduzir alterações após a esta mesma data (pelo menos até que o formador clone o repositório). O repositório deve seguir a organização descrita no ponto anterior.

3. Relatório em **PDF** - ficheiro `relatorio.pdf` - cuja estrutura e formatação deverão ser a do modelo fornecido em anexo. Em termos de formatação adapte apenas a designação da acção e o nome dos módulos. Siga as recomendações relacionadas com a elaboração de um relatório dadas pelo formador Fernando Ruela. O relatório deve incluir uma capa simples com o símbolo do IEFP, referência ao Centro de Formação de Alcântara, data, indicação do curso e da acção (eg, Técnico de Informática - Sistemas 07) e dos elementos que elaboraram o trabalho.

Em termos de conteúdo o seu relatório deve possuir as seguintes secções e anexos:

3.1.1 Introdução e Objectivos (não plagiar a deste enunciado!).

3.1.2 Desenho e Estrutura que deverá conter:

- Para os três últimos efeitos, um fluxograma com o algoritmo geral desse efeito especial. Assuma que todo o texto a exibir é uma variável de entrada do fluxograma. Aqui pode ignorar a questão da concatenação. Assuma também que esse texto é uma sequência de caracteres, tal como uma string em Python.

Um dos três fluxogramas seguintes:

- **vcypher**: Um fluxograma a ilustrar o algoritmo geral de encriptação do vcypher.

- **treep**: Um fluxograma a ilustrar o algoritmo para gerar a àrvore de directorias e ficheiros do treep.

- **syms**: Um fluxograma a descrever o algoritmo de detecção de duplicados por contéudo por binário

3.1.3 Implementação. A secção Implementação deve indicar aspectos de implementação que considere relevantes. É aqui que deve indicar a "docstring" utilizada pelo docopt (caso tenha utilizado esta biblioteca). Deve indicar para que plataforma desenvolveu e testou o seu programa, a versão do compilador e das bibliotecas utilizadas.

É também aqui que deve dar instruções sobre como "compilar" o seu programa e produzir um executável (num programa maior poderia ser pertinente criar uma secção Compilação e Instalação).

É também nesta secção que deve indicar que elementos opcionais foram implementados e como.

3.1.4 Conclusão. Além de seguir as recomendações sobre a elaboração de uma conclusão de relatório, deve também listar o que foi implementado e o que ficou por implementar.